

WEB APPLICATION

PENETRATION TESTING LAB REPORT

SQL Injection — UNION-Based Attack

Determining the Number of Columns Returned by a Query

Prepared by:	Mohammed Ashraf Mokhtar
Role:	Penetration Tester / Security Researcher
Target Application:	PortSwigger Web Security Academy — "SQL injection UNION attack, determining the number of columns returned by the query" (Practitioner Lab)
Testing Type:	Authorized Web Application Penetration Test (Training Lab Environment)
Report Date:	September 2, 2026
Lab Status:	SOLVED

LinkedIn: <https://www.linkedin.com/in/1mohammed-ashraf/>

GitHub: <https://github.com/moashraf2006>

Table of Contents

- 1. Executive Summary
- 2. Engagement Scope
- 3. Methodology
- 4. Vulnerability Finding: SQL Injection (UNION-Based, Column Enumeration)
 - 4.1 Summary
 - 4.2 Technical Details & Steps to Reproduce
 - 4.3 Evidence
 - 4.4 Impact
 - 4.5 Remediation
- 5. Conclusion

1. Executive Summary

This report documents a web application security assessment performed against a deliberately vulnerable e-commerce application ("We Like To Shop") hosted on PortSwigger's Web Security Academy training platform. The objective of the engagement was to identify and exploit a SQL Injection vulnerability present in the product category filter functionality, and to demonstrate the first stage of a UNION-based SQL injection attack: determining the number of columns returned by the underlying database query.

The assessment confirmed that the category filter parameter is directly concatenated into a backend SQL query without adequate input validation or parameterization. This allowed a tester-supplied ORDER BY clause and, subsequently, a UNION SELECT statement to be injected and executed by the database. Through iterative testing, the exact column count of the vulnerable query (three columns) was successfully identified, satisfying the lab's success criteria and providing the foundation required for further data extraction via UNION-based SQL injection.

The vulnerability is rated Critical, consistent with industry-standard classification of SQL injection flaws (OWASP Top 10 – A03:2021 Injection), as it can lead to unauthorized access, modification, or exfiltration of backend database contents, including sensitive user and application data, if exploitation were extended beyond this proof of concept.

Key Findings

Finding	Severity	Status
SQL Injection in 'category' parameter — UNION-based column enumeration	Critical	Confirmed / Exploited

2. Engagement Scope

Target	Oac20071034f073780c3356b009500a6.web-security-academy.net
Application	"We Like To Shop" demo e-commerce store (PortSwigger lab instance)
Vulnerable Endpoint	GET /filter?category={value}
Testing Window	September 2, 2026 (single session)
Tools Used	Web browser (manual URL manipulation / address bar), PortSwigger Web Security Academy lab environment
Authorization	Testing was performed exclusively within PortSwigger's authorized, purpose-built training lab environment

This engagement was limited to a single, self-contained training lab provided by PortSwigger's Web Security Academy. No production systems, third-party infrastructure, or real user data were involved. The scope was restricted to the 'Determining the number of columns required' lab within the SQL Injection learning path, which follows on from the general SQL Injection UNION attacks module.

3. Methodology

The assessment followed a structured, repeatable methodology consistent with standard black-box web application testing practice:

- Reconnaissance — Reviewed the lab brief and target application to identify the vulnerable feature (the product category filter) and confirm that query results were reflected in the HTTP response, a prerequisite for a UNION attack.
- Injection Point Identification — Located the 'category' GET parameter on the /filter endpoint as the injectable input.
- Column Count Enumeration (ORDER BY method) — Submitted an incrementing series of ORDER BY clauses (ORDER BY 1, 2, 3, 4 ...) to determine, by observing successful responses versus SQL errors, how many columns the underlying SELECT query returns.
- Confirmation (UNION SELECT method) — Once the error threshold was identified, submitted a UNION SELECT statement with the corresponding number of NULL placeholders to confirm the exact column count and validate that the injected query executed successfully.
- Validation — Verified successful exploitation via the application's lab status indicator, which changed from 'Not solved' to 'Solved'.

All payloads were submitted directly via the browser address bar as URL-encoded query string values, requiring no additional tooling such as an intercepting proxy.

4. Vulnerability Finding: SQL Injection (UNION-Based, Column Enumeration)

4.1 Summary

Vulnerability Class	SQL Injection (CWE-89)
OWASP Category	A03:2021 — Injection
Affected Parameter	category (GET, /filter endpoint)
Severity	Critical
CVSS 3.1 (indicative)	9.8 (Critical) — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The application's product category filter passes user-supplied input directly into a backend SQL query without sanitization or the use of parameterized statements. Because the results of this query are rendered back into the page, an attacker can leverage a UNION SELECT injection to append arbitrary data from other database tables to the displayed results — but only once the exact number of columns returned by the original query is known. This finding documents the successful enumeration of that column count.

4.2 Technical Details & Steps to Reproduce

Step 1 — Baseline request. The category filter was tested with a single quote to break out of the original SQL string, followed by an ORDER BY clause and a comment sequence to neutralize the remainder of the original query:

- Payload: ' ORDER BY 1--
- URL: /filter?category='+ORDER+BY+1--

Step 2 — Incremental enumeration. The ORDER BY index was increased one at a time (2, then 3, then 4) and the application's response was observed after each request:

- ORDER BY 2-- → page rendered normally (HTTP 200)
- ORDER BY 3-- → page rendered normally (HTTP 200)
- ORDER BY 4-- → application returned an Internal Server Error, indicating the query does not have a fourth column to sort by

This confirmed the underlying SELECT statement returns exactly three columns.

Step 3 — Exploitation / confirmation. A UNION SELECT statement with three NULL placeholders (matching data types are not required for NULL) was submitted to confirm successful query injection:

- Payload: ' UNION SELECT NULL,NULL,NULL--
- URL: /filter?category='+UNION+SELECT+NULL,NULL,NULL--

The application returned an HTTP 200 response and rendered the injected row, and the lab status indicator updated to 'Solved', confirming successful exploitation.

4.3 Evidence

The following screenshots document each stage of the testing process, from initial lab access through to successful exploitation.

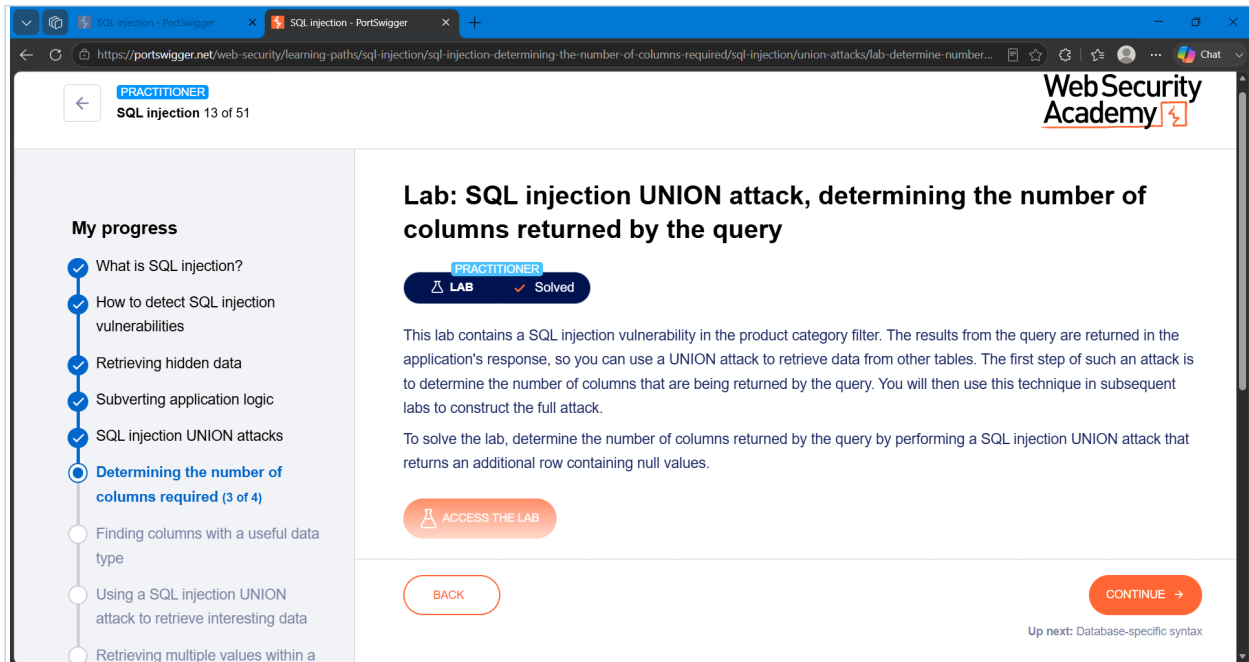


Figure 1 — PortSwigger lab brief: "SQL injection UNION attack, determining the number of columns returned by the query."

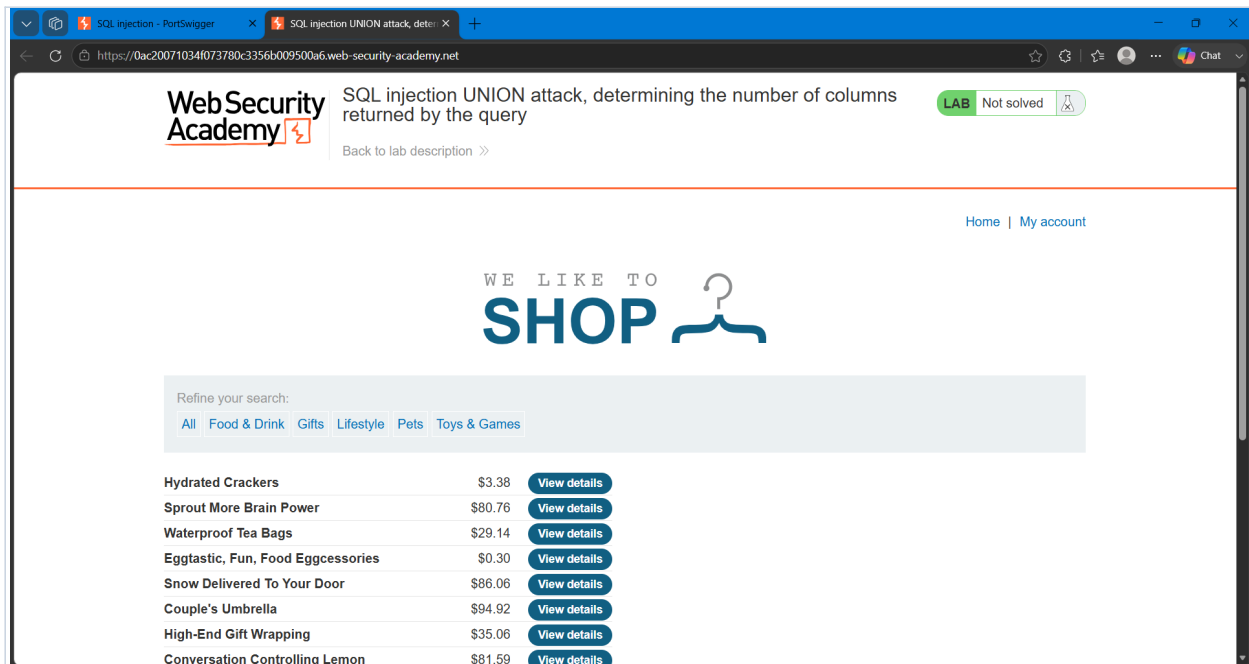


Figure 2 — Target application ("We Like To Shop") in its initial, unauthenticated state; lab status shows "Not solved."

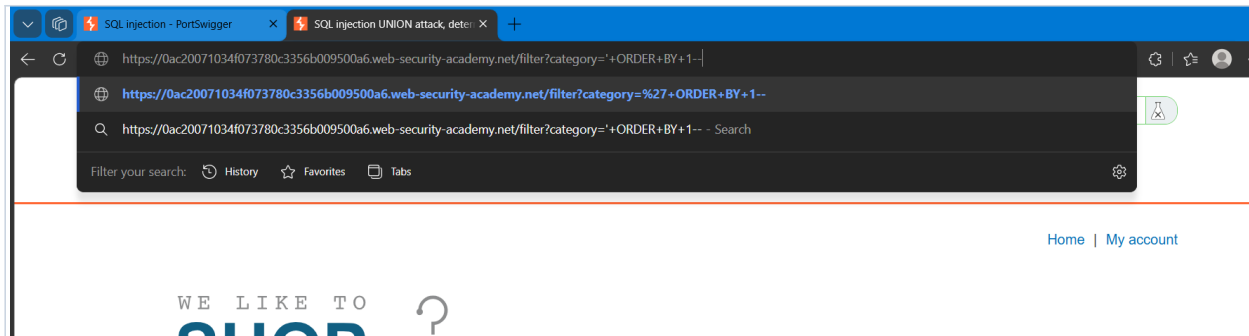


Figure 3 — Baseline injection payload entered in the browser address bar: `category='+ORDER+BY+1--`



Figure 4 — `ORDER BY 1` executes successfully; no error returned, confirming injection point is reachable.

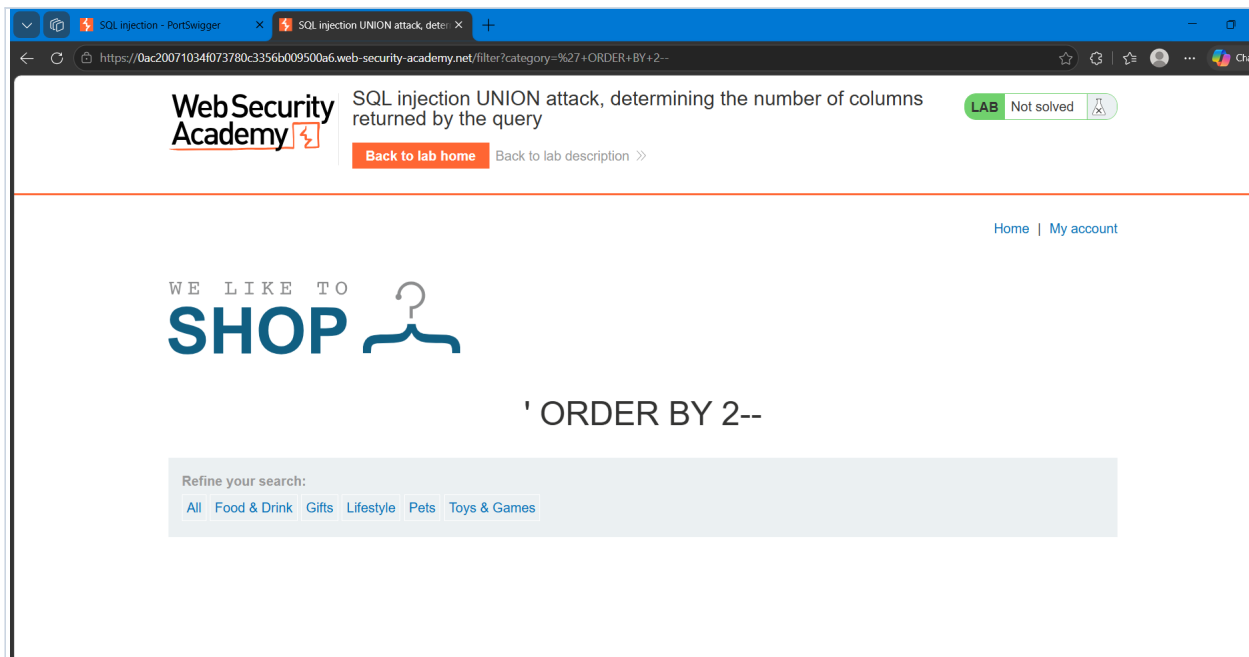


Figure 5 — ORDER BY 2 executes successfully with no SQL error.

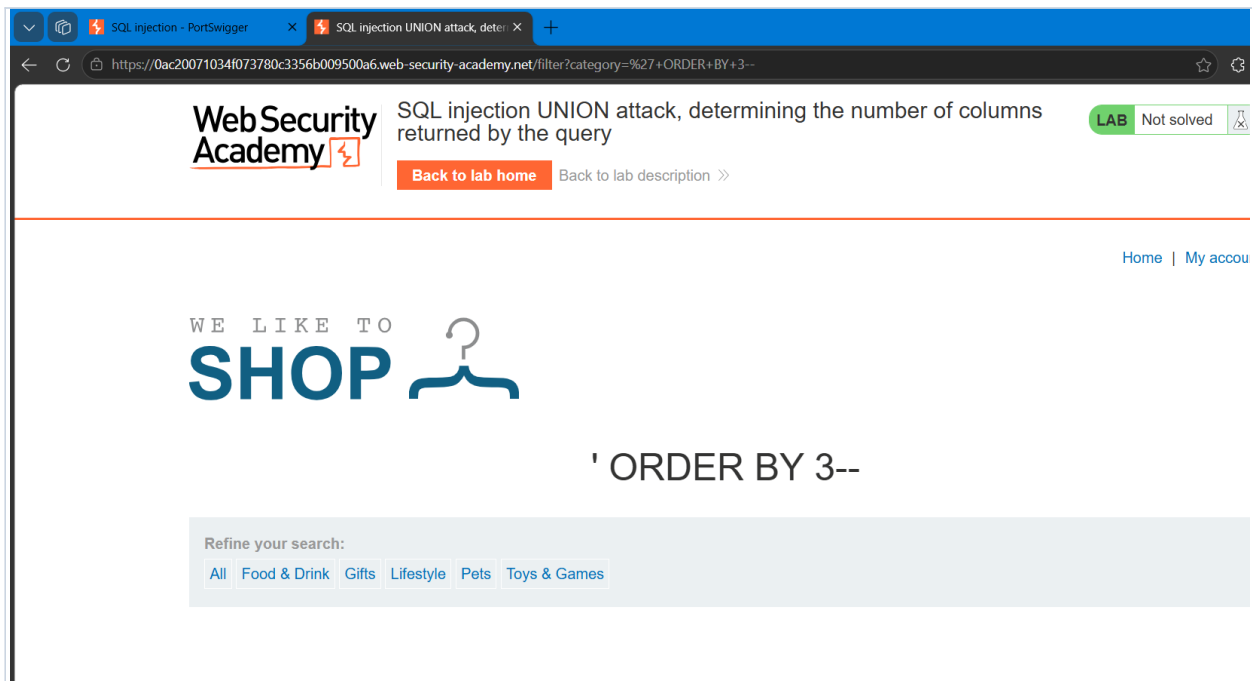


Figure 6 — ORDER BY 3 executes successfully with no SQL error.

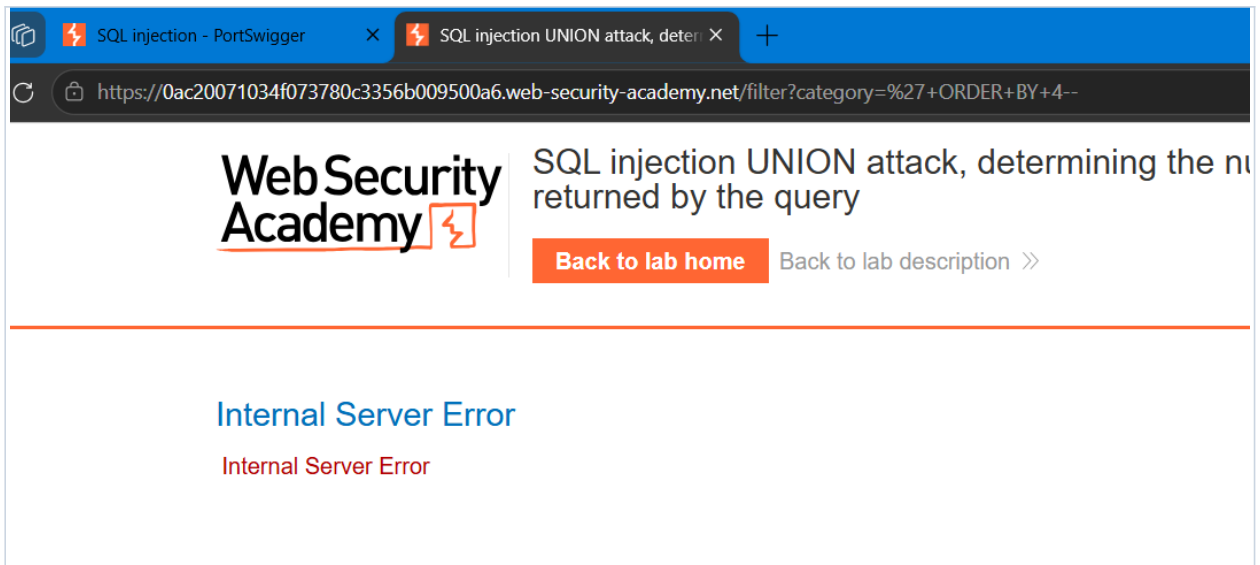


Figure 7 — ORDER BY 4 triggers an Internal Server Error, indicating the query returns only 3 columns.

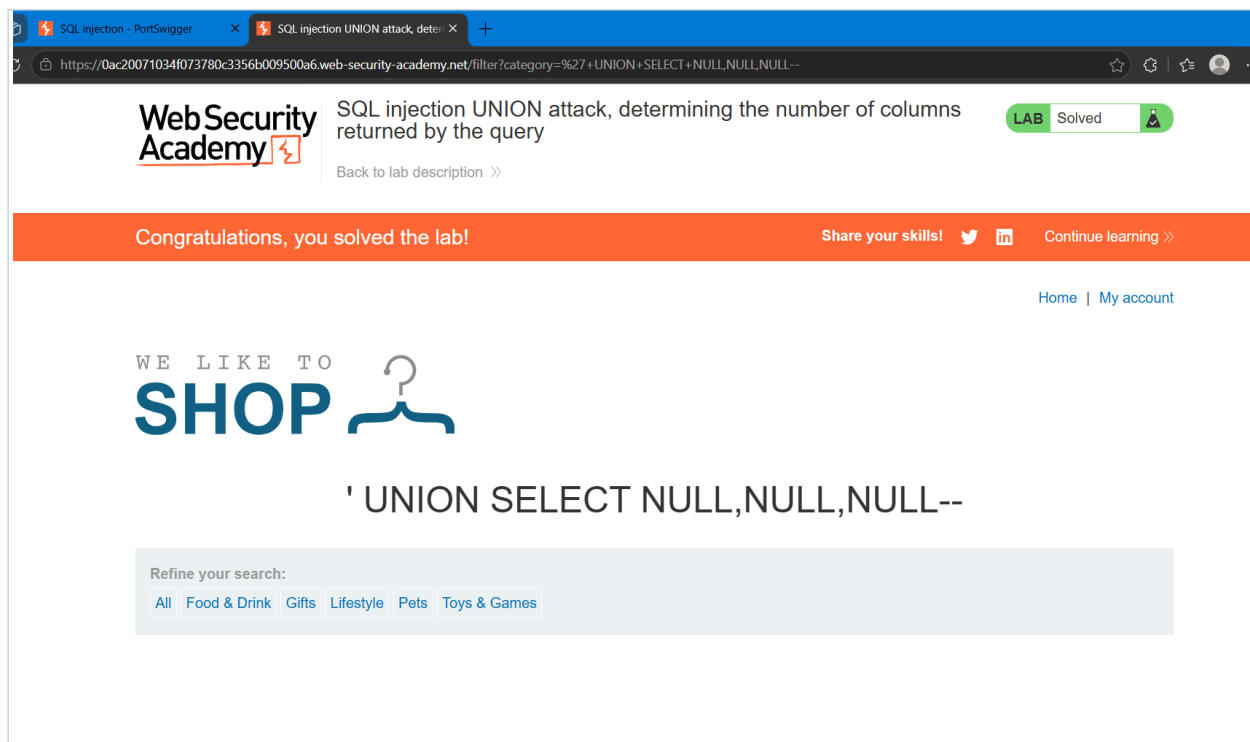


Figure 8 — UNION SELECT NULL,NULL,NULL successfully injected; lab status updates to "Solved," confirming exploitation.

4.4 Impact

Although this proof of concept was limited to enumerating the query's column count, the same injection point can be leveraged to append a UNION SELECT statement retrieving arbitrary data from any accessible database table — including, in a real-world equivalent, customer records, authentication credentials, session tokens, or payment information. Because the vulnerable parameter is unauthenticated and reachable via a simple GET request, exploitation requires no special privileges and could be automated at scale. In a production environment, this class of vulnerability routinely leads to full database compromise, data breach, and downstream account takeover.

4.5 Remediation

- Use parameterized queries / prepared statements (or a vetted ORM) for all database access — never concatenate user-supplied input directly into SQL statements.
- Apply strict server-side allow-list validation to the category parameter, restricting it to known, expected values (e.g., a fixed set of category IDs or names).
- Enforce the principle of least privilege on the database account used by the application, limiting its ability to access tables and columns outside what the application legitimately requires.
- Disable verbose/default error messages (e.g., raw "Internal Server Error" / stack traces) in production, replacing them with generic error responses to avoid leaking information useful for injection point discovery.
- Deploy a Web Application Firewall (WAF) as a defense-in-depth measure, and incorporate automated SAST/DAST scanning for injection flaws into the CI/CD pipeline.

5. Conclusion

This engagement successfully identified and exploited a SQL Injection vulnerability in the target application's product category filter, confirming that the underlying database query returns three columns and validating this via a working UNION SELECT injection. The lab's success criteria were met, and the finding was reproduced end-to-end with full evidentiary screenshots. This vulnerability class remains one of the most severe and common risks facing web applications, and the remediation steps outlined above — principally the adoption of parameterized queries — should be treated as mandatory for any application handling untrusted user input.

Report prepared by Mohammed Ashraf Mokhtar.